Closed-Loop Threat-Guided Auto-Fixing of Kubernetes Container Security Misconfigurations

Brian Mendonca and Vijay K. Madisetti, *Fellow, IEEE*

Abstract—Containerized applications depend on Kubernetes YAML manifests that configure images, permissions, and storage; small configuration errors can introduce outages or security gaps. Most existing tools detect or block such problems but do not produce verified, ready-to-apply repairs. This paper studies `k8s-auto-fix`, a closed-loop remediation system that detects an issue, proposes a small patch, verifies it against Kubernetes-specific acceptance gates, and queues the accepted patch for review or application. On a fixture-seeded Kind/Kubernetes 1.34.0 replay with placeholder images and seeded service accounts, all 1,000 verifier-accepted patches passed both server-side dry-run and live apply checks. On a 15,718-detection offline run, the system automatically fixed 13,338 detections overall (auto-fix rate 0.8486); among the 13,373 attempted patches, deterministic rules plus safety checks accepted 13,338 (99.74%; median patch ops 9). An optional LLM mode reaches 88.52% acceptance on a 5,000-manifest corpus. In deterministic queue replay, a risk-aware scheduler reduces P95 wait for high-risk items by 7.9×. The released artifact contains the data and scripts used for these results.

Index Terms—Kubernetes, SAST, DAST, Admission control, Server-side dry-run, YAML, Pod Security, JSON Patch, Policy Enforcement, Kyverno, OPA Gatekeeper, Auto-fix, CI/CD, KEV-style risk, Guidance retrieval, Risk-based scheduling

1 Introduction

Kubernetes is now a common substrate for cloud services, yet its declarative configuration model remains prone to security misconfigurations [32], [38]. A manifest is the text file that tells Kubernetes what to run, with which permissions, and on what resources. When these files are edited by hand, a stray `privileged: true`, a floating `:latest` image tag, or a missing `runAsNonRoot` line can quietly open a security gap or break a deployment.

Problem. Existing tooling addresses only part of this lifecycle. Static analyzers and admission controllers such as Kyverno and OPA Gatekeeper are effective at *detecting* or *blocking* insecure manifests, but they rarely produce a safe, ready-to-apply repair for an existing, non-compliant resource. Operators are therefore left to remediate by hand, which is slow and error-prone. Recent advances in Large Language Models (LLMs) suggest a path to automated repair and configuration validation [20], [34], but security studies show that LLM reasoning over vulnerabilities remains unreliable without external checks [31]. The open problem we target is *validated* remediation: turning a detected misconfiguration into a small patch that is verified against Kubernetes-specific safety and admission gates before it is allowed near a cluster.

- *B. Mendonca is with the College of Computing, Georgia Institute of Technology, Atlanta, GA 30332 USA (e-mail: brian.mendonca6@gmail.com).*
- *V. K. Madisetti is with the School of Cybersecurity and Privacy, Georgia Institute of Technology, Atlanta, GA 30332 USA (e-mail: vkm@gatech.edu).*
- *Corresponding author: Vijay K. Madisetti.*

Approach. We present `k8s-auto-fix`, a closed-loop *detect* → *propose* → *verify* → *prioritize* pipeline whose proposer defaults to deterministic rules and admits optional LLM-generated patches only behind the same guardrails. The design is *threat-guided* in two ways. First, the verifier enforces a fixed set of security invariants—a policy re-check that the original violation is gone, Kubernetes API admission through server-side dry-run, and universal no-new-violation safety gates—so the threat a detection represents must be demonstrably removed before a patch is accepted. Second, the scheduler orders accepted work by a transparent risk score R built from policy severity, configured KEV-style boosts, and observed verifier priors; live CVE/EPSS feed joins are planned enrichment rather than required for the reported results. This acceptance contract is narrow: a patch is accepted only when it is policy-clean, API-admissible, and free of the universal safety regressions checked by the verifier.

Contributions. This paper makes the following contributions:

- A closed-loop remediation pipeline for Kubernetes manifests that couples deterministic and optional LLM-based patch generation with a multi-gate verifier (policy re-check, Kubernetes API admission through server-side dry-run, and universal safety invariants), so every accepted patch carries machine-checkable evidence that it removed the violation without introducing a new one.
- A threat-guided, risk-aware scheduler that prioritizes accepted patches by policy severity, configured KEV-style boosts, and observed verifier priors while an aging term bounds starvation, turning a flat remedi-

ation backlog into a risk-ordered queue.
- A reproducibility bundle for deterministic and LLM-backed modes on offline corpora and a fixture-seeded live cluster, with paper-facing summary tables regenerated by `make reproducible-report` and per-claim command scope enumerated in ARTIFACTS.md.

Organization. Section 2 defines the task, acceptance boundary, and threat model. Section 3 presents the closed-loop design, including the verifier and scheduler. Section 4 describes the implementation and artifacts. Section 5 evaluates acceptance, latency, risk reduction, and patch size. Section 6 reports verifier ablations and failure taxonomies. Section 7 compares against prior systems, Section 8 states limits, and Section 9 concludes.

2 Task and System Model

2.1 Model and Acceptance Predicate

We model a manifest as a JSON/YAML document m and a detection as a tuple $d = (m, \pi, v)$, where π is the violated policy identifier and v is the violation evidence emitted by the detector. The proposer produces a JSON Patch δ (RFC 6902 [8]), and applying it yields the patched manifest $m' = \delta(m)$. A detection is *resolved* when m' satisfies the acceptance predicate $\mathrm{Acc}(m', \pi) = \mathrm{policy}(m', \pi) \wedge \mathrm{schema}(m') \wedge \mathrm{dryrun}(m') \wedge \mathrm{safe}(m')$, i.e. the conjunction of the four verifier gates defined in Section 3. The scheduler then ranks resolved items using a risk score R_i for queue item i, an empirical verifier success probability p_i for its policy, the observed proposer+verifier latency $\mathbb{E}[t_i]$, the accumulated queue age wait_i, and a configured KEV-style boost kev_i when the policy is flagged in the current artifact. Unless otherwise noted, wait times are reported in hours and fairness statistics (Gini, starvation rate) are computed over these waits. Section 3 gives the closed forms of R_i and the scheduling score S_i.

Detector disagreement and false positives. The detector runs `kube-linter` and a policy engine (Kyverno/OPA) and takes the *union* of their findings; a patch must satisfy *both* engines during verification, so the union is conservative for recall while the verifier remains the precision boundary. False positives are handled structurally rather than heuristically: every proposer fix is guarded by JSON Pointer existence checks, so a finding that does not correspond to a real field produces no edit (an empty patch) rather than a spurious change, and the policy re-check gate $\mathrm{policy}(m', \pi)$ rejects any patch that does not actually clear the asserted violation. A detection the proposer cannot resolve under these guards is never silently "fixed"; it is queued for human review.

Attempts and retry budget. An *attempt* is one proposer→verifier cycle for a single detection: the proposer emits a candidate patch and the verifier evaluates $\mathrm{Acc}(m', \pi)$. If an attempt fails, the failing gate and error trace are fed back and the proposer may retry, up to a configurable budget `max_attempts` (default 3, set in `configs/run.yaml`). After the budget is exhausted the item is dropped from automated handling and routed to review. Per-attempt latency and outcome are logged to `data/policy_metrics.json`, which supplies the online estimates p_i and $\mathbb{E}[t_i]$ the scheduler consumes alongside KEV flags.

2.2 Threat Model

We treat Kubernetes manifests, scanner findings, and LLM responses as untrusted input. Trusted components include the detector/verifier binaries, the scheduler, and the per-cluster fixtures under infra/fixtures/; these run inside the CI environment we control and write the artifacts cited throughout Section 5. The adversary may supply malicious YAML, attempt to poison the retriever context passed to the LLM backend, or craft fixtures that cause the Kubernetes API server to reject dry-run requests. We do not defend against compromised detector binaries, forged audit logs, or supply-chain attacks that deliver malicious container images—those threats are out of scope here and would be handled by image-signing and SBOM enforcement layers. Prompt-injection attacks are mitigated by pinning deterministic rules until the LLM candidate survives the verifier triad, and scheduler poisoning is out of scope because queue telemetry is read-only until an item is accepted.

2.3 Threats and Mitigations

The reproducibility bundle (`make reproducible-report`) regenerates Table 4 directly from JSON artifacts so that every metric can be audited. Semantic regression checks now block Grok-generated patches that remove containers or volumes, and fixtures under infra/fixtures/ seed RBAC/NetworkPolicy gaps before verification. We threat-modeled malicious or placeholder manifests: the guidance retriever limits prompt context to policy-relevant snippets, the verifier enforces policy and `kubectl` API-admission gates, and the scheduler never surfaces unverified patches. Residual risks—primarily infrastructure assumptions and LLM hallucinations—are characterized in the failure taxonomies (Tables 5 and 6) and triaged before publication. A privileged-DaemonSet case study in the artifact bundle shows the guardrails preserving required host integrations while removing privilege escalation paths.

Secret hygiene is enforced end-to-end: the proposer replaces secret-like environment values with `secretKeyRef` references, sanitizes generated names, and documents the enforced invariants in docs/security_considerations.md.

3 System Design

We implement the closed loop *Detector* → *Proposer* → *Verifier* → *Scheduler* shown in Figure 1. Detectors produce structured JSON findings; the proposer applies rule-based guards (with optional LLM backends) to emit minimal JSON Patches; the verifier enforces policy re-checks, `kubectl apply -dry-run=server` API admission, and safety invariants; and the scheduler orders work using risk-aware priority scoring with UCB-style exploration. Each stage persists artifacts (detections, patches, verified outcomes, queue scores), enabling reproducible evaluation (Section 5).

3.1 Pipeline and Verifier Gates

The workflow consists of four stages: the **Detector** ingests a Kubernetes manifest and uses both `kube-linter` and a policy engine (Kyverno/OPA) to identify violations, taking the union of all findings; the **Proposer** takes the manifest and violation data and generates a JSON Patch, defaulting to deterministic rules for the covered policy families (image tags, privilege, capabilities, non-root execution, read-only root filesystems, host namespace/mount controls, seccomp, and resource requests/limits) while guarding each operation with JSON Pointer existence checks and enforcing minimality/idempotence via `tests/test_patch_minimality.py`; the **Verifier** applies the patch to a copy of the manifest and subjects it to the verification gates described below, recording evidence in `data/verified.json`; and the **Budget-aware Retry** loop uses a configurable retry budget (`max_attempts` in `configs/run.yaml`, default 3) so the proposer can re-attempt if verification fails, logging the error trace for inspection.

The verifier accepts a patched manifest only after four gates pass. The **Policy Re-check** re-evaluates the patched manifest with the same policy logic that triggered the violation, using explicit assertions for each covered policy (e.g., `no_latest_tag`, `no_privileged`, `drop_capabilities`, `drop_cap_sys_admin`, `run_as_non_root`, `read_only_root_fs`, `no_host_*` flags, `no_allow_privilege_escalation`, `enforce_seccomp`, `set_requests_limits`); the detector hook for re-scanning is available via `-enable-rescan`, and non-allowlisted `hostPath` volumes can be rewritten to `emptyDir: {}` by guardrails to satisfy the universal safety gate. Because that rewrite can change workload storage semantics, accepted `hostPath` rewrites are marked for human review unless workload requirements are known. The **Schema Validation** step applies the JSON Patch via `jsonpatch` and rejects malformed paths or operations, surfacing issues to the retry loop. The **Server-side Dry-run** uses `kubectl apply -dry-run=server`, when available, to simulate how the Kubernetes API server would handle the change, marking failures as not accepted and persisting CLI output for analysis. The **No-New-Violations Safety Gates** enforce universal security assertions to prevent regressions: they block `privileged: true` in any container; reject `capabilities.add` entries containing `NET_RAW`, `NET_ADMIN`, `SYS_ADMIN`, `SYS_MODULE`, `SYS_PTRACE`, or `SYS_CHROOT`; flag empty `capabilities.drop` lists when capabilities are present; require each container to specify a non-empty image; and restrict host mounts to approved paths (`/var/run/secrets/kubernetes.io/serviceaccount`, `/var/lib/kubelet/pods`, `/etc/ssl/certs`), rewriting non-allowlisted `hostPath` volumes to `emptyDir: {}` by guardrails and routing those semantic edits to review when requirements are unknown.

Scheduler example. In a queue with a privileged container, a ':latest' image, and a low-risk resource-limit item, the privileged item is served first; as the others wait, the aging term raises their score. This gives high-risk work priority while bounding starvation, a fairness target shared with constrained bandit formulations [28].

3.2 End-to-End Walkthrough on Real Manifests

To make the closed-loop pipeline concrete, we trace two real-world manifests from the repository's test suite through detection, patching, verification, and scheduling.

Case 1: Remediating a Privileged Pod with a `:latest` Image Tag

This example, drawn from data/manifests/001.yaml, shows a common but high-risk pattern: a privileged container using a floating tag.

```
apiVersion: v1
kind: Pod
spec:
  containers:
  - name: app
    image: acme/api:latest
    securityContext:
      privileged: true
      allowPrivilegeEscalation: true
      capabilities: { add: ["SYS_ADMIN", "NET_ADMIN"] }
```

Walkthrough. We scan the file, draft a small idempotent fix, run policy, server-dry-run/API-admission, and safety gates, and only then queue it. "Accepted" means policy- and API-admissible, not a guarantee of runtime workload correctness.

1. Detect (Union): The detector reports `no_privileged`, `drop_capabilities`, `run_as_non_root`, and `no_latest_tag` in a structured detection record.

2. Propose (Rules Engine): The proposer emits a minimal JSON Patch. The pinned tag (`acme/api:1.42.0`) is illustrative: the deterministic fixer requires an operator-configured replacement tag or digest for `no_latest_tag` and otherwise routes the item to review.

```
[
 {"op":"replace","path":"/spec/containers/0/
    securityContext/privileged","value":false},
 {"op":"replace","path":"/spec/containers/0/
    securityContext/allowPrivilegeEscalation","value":
    false},
 {"op":"remove","path":"/spec/containers/0/securityContext
    /capabilities/add"},
 {"op":"add","path":"/spec/containers/0/securityContext/
    capabilities/drop","value":["ALL"]},
 {"op":"add","path":"/spec/containers/0/securityContext/
    runAsNonRoot","value":true},
 {"op":"replace","path":"/spec/containers/0/image","value
    ":"acme/api:1.42.0"}
]
```

3. Verify (Triad): The verifier applies the patch to an in-memory copy, confirms the original policies are cleared, validates the Kubernetes schema, and succeeds under `kubectl apply -dry-run=server` in a fixture-seeded Kind cluster. The successful outcome is recorded in the verifier log.

4. Schedule (risk-aware priority): Because the item is high risk, historically fixable, and quick to verify, it receives a high score and moves earlier in the queue.

Case 2: Hardening a Worker Pod with a `hostPath` Mount

```
spec:
  containers:
  - name: worker
    securityContext: { readOnlyRootFilesystem: false }
    resources: {}
  volumes:
  - name: host
    hostPath: { path: "/var/run/docker.sock" }
```

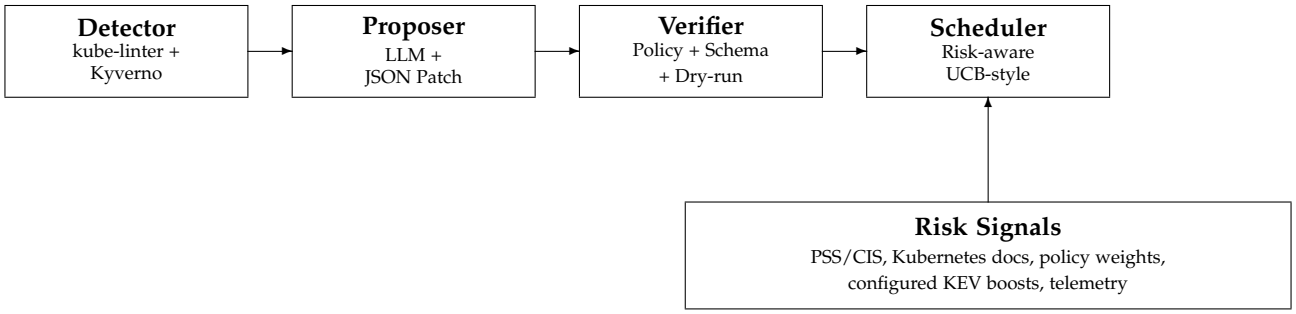


Figure 1. Closed-loop architecture. Candidate patches from deterministic or LLM-backed proposers are accepted only after policy re-check, server-side dry-run/API admission, and no-new-violation gates; accepted items then enter the risk-aware scheduler.

This second case, from data/manifests/002.yaml, targets a writable root filesystem, a dangerous `hostPath`, and missing resource requests/limits.

1–2. Detect and propose: The detector flags `read_only_root_fs`, `no_host_path`, and `set_requests_limits`; the rules engine hardens the filesystem, proposes replacing the disallowed `hostPath` with `emptyDir`, and adds resource quotas:

```
[
 {"op":"replace","path":"/spec/containers/0/
    securityContext/readOnlyRootFilesystem","value":true},

 {"op":"remove","path":"/spec/volumes/0/hostPath"},
 {"op":"add","path":"/spec/volumes/0/emptyDir","value
    ":{}},
 {"op":"add","path":"/spec/containers/0/resources","value
    ":{"requests":{"cpu":"100m","memory":"128Mi"},"limits
    ":{"cpu":"500m","memory":"256Mi"}}}
]
```

3–4. Verify and schedule: The verifier confirms the `no_host_path` assertion is gone, the YAML remains valid, and server dry-run accepts the object. Allowlisted host mounts would be preserved, and the policy re-check discipline is the same mechanism that catches the ablation escapes in Section 5. Because replacing a host mount can change workload semantics, this accepted patch is review-required unless the workload's storage requirement is known. The item receives a moderate score and is scheduled behind higher-risk work.

Comparison with existing approaches. Kyverno mutation focuses on admission-time defaults and depends on cluster fixtures and policy/controller context for complex hardening. GenKubeSec localizes and explains issues but leaves remediation and validation manual, and LLMSecConfig generates LLM repairs with scanner checks but lacks our triad's server-side dry-run and hard safety invariants.

Acceptance boundary and scheduling. The boundary matrix shows that weaker acceptance predicates admit 312–551 records rejected by the corresponding full-verifier snapshots on the shared 5k records (Table 7); live-cluster replay recorded 100% server-side dry-run and live apply acceptance on the fixture-seeded subset. The replay-based scheduler prioritizes risk (P95 wait from 102.3 h to 13.0 h), closing the highest-impact items first under bounded budgets.

Table 1 makes the lifecycle gap explicit: comparable systems can detect, mutate, or suggest repairs, but they differ sharply on the evidence an operator receives before trusting a patch. We use "safety invariants" below for universal no-new-violation gates beyond confirming that one scanner finding disappeared.

3.3 Risk Scoring

We rank accepted fixes by policy risk and expected verifier cost. The current results use policy weights, configured KEV-style flags, and observed success/latency priors; public CVE, EPSS, and KEV feed joins are planned enrichment.

The scheduler consumes data/policy_metrics.json, which stores per-policy priors for success probability, expected latency, KEV flags, and baseline risk. The calibration pass (data/risk/policy_risk_map.json) augments those priors with observed detection/resolution counts, while data/risk/risk_calibration.csv captures corpus-level ΔR and residual risk (Table 3). Future iterations will enrich each queue item with container-image CVE joins via `Trivy` and `Grype` [15], [16], CVSS/EPSS feeds [12], [14], and CISA KEV catalog checks [13] so that R reflects both exposure (Pod Security level, dangerous capabilities, host mounts) and exploit likelihood. The risk score R then feeds the scheduler scoring function, allowing us to report absolute risk and per-patch risk reduction ΔR directly.

3.4 Guidance Refresh and Retrieval Hooks

We curate policy guidance under docs/policy_guidance/raw/; scripts/refresh_guidance.py now refreshes Pod Security, CIS, and Kyverno snippets [1], [2], [23] (backed by docs/policy_guidance/sources.yaml) to keep guardrails current. LLM-backed proposer modes can retrieve these snippets at prompt time, and the roadmap extends this into a full RAG loop: chunk guidance with metadata (policy family, resource kind, field path, image→CVE), cache recent verifier failures, and retrieve targeted passages when retries occur. This keeps the prompt budget bounded while grounding fixes in up-to-date hardening language.

3.5 Risk-Aware Scheduler with UCB-Style Exploration

Overview. The scheduler is a risk-aware priority queue: higher-risk items are scheduled first, while an aging term raises the priority of long-waiting items to prevent starvation. **Notation.** R_i denotes the risk units for item i; p_i

Table 1
Lifecycle contract for remediation tools: what evidence exists before an operator can trust a patch.

Lifecycle question	k8s-auto-fix (this work)	Kyverno	GenKubeSec	LLMSecConfig
When does it act?	Post-hoc on stored manifests and replayed live resources	Admission-time create/update path	Offline localization and explanation	Offline LLM repair attempt
What patch artifact exists?	Minimal RFC 6902 JSON Patch plus verifier evidence	Mutation policy result when a policy is installed	Textual remediation guidance	YAML edit proposed by the model
What accepts the patch?	Policy re-check + `kubectl` server dry-run/API admission + safety invariants	Admission controller outcome in the target cluster	Human/manual validation	Scanner re-checks; no server dry-run or hard safety invariants
How is work ordered?	Risk-aware score $Rp/\mathbb{E}[t]$ with aging and KEV-style boosts	Admission arrival order	None	None

is its empirical verifier success probability; $\mathbb{E}[t_i]$ is the observed proposer+verifier latency; wait_i tracks queue age; kev_i equals the configured KEV-style boost when the policy is flagged in the current artifact (otherwise 0); and ε is a small positive floor preventing division by zero.

$$S_i = \frac{R_i \cdot p_i}{\max(\varepsilon, \mathbb{E}[t_i])} + \text{explore}_i + \alpha\,\text{wait}_i + \text{kev}_i \quad (1)$$

The risk construction is explicit. Each detection maps to a policy identifier; we pull the static weight w_{policy} from `data/risk/policy_risk_map.json` and add the configured KEV-style surcharge κ when the policy is flagged in `data/policy_metrics.json`. The empirical success prior remains the separate p_i term in Equation 1; it is not an EPSS feed value in the current artifact. Formally,

$$R_i = w_{\text{policy}} + \kappa \cdot \mathbf{1}_{\text{configured KEV}},$$

with $\kappa = 25$ risk units in the current configuration. p_i is the on-line verifier pass rate for that policy (accepted / attempted counts in `data/policy_metrics.json`), and $\mathbb{E}[t_i]$ is the running average of proposer+verifier latency recorded in the same file. We also report $\Delta R_i = R_i - R_i^{\text{post}}$ for every accepted patch, summing per corpus to produce Table 3. The supplemental appendix provides a numeric worked example.

Equation 1 mirrors UCB-style bandit heuristics [27]. p_i and $\mathbb{E}[t_i]$ are refreshed from proposer/verifier telemetry; exploration uses an upper-confidence term and aging ensures fairness. Each loop iteration scores the queue, proposes and verifies the highest-scoring item, updates the online pass-rate and latency estimates from the verifier callback, and increments wait time for the remaining items. The evaluation in Section 5 contrasts this risk-aware scheduler against FIFO, measuring reductions in top-risk wait time. Future work will incorporate additional risk signals (EPSS, CVSS) and batch-aware policies; the present heuristic is evaluated below.

4 Implementation

The implementation follows the design above but keeps the default reproducible path independent of external LLM APIs. A three-stage command-line pipeline records detections, candidate patches, verifier outcomes, and scheduler scores as JSON/CSV artifacts; optional LLM-backed modes use the same verifier boundary. The released artifact maps each paper-facing claim to source files and regeneration commands in `ARTIFACTS.md`, while the main text focuses on the interfaces that affect the evaluation.

Reproducing the results. The pipeline runs as `make detect`, `make propose`, and `make verify`; `make reproducible-report` regenerates summary tables from checked-in JSON/CSV outputs. Full-corpus regeneration (15,000+ detections) takes roughly 20–30 minutes; the dependency snapshot is `data/repro/environment.json`.

Scalability and integration. The pipeline sustains millisecond-scale proposer and sub-second verifier latency on the supported corpus (Table 4); the scheduler replays thousands of queue items without recomputing detections. Rules, LLM, and replay modes all cross the same acceptance boundary.

4.1 Detection Records and Test Evidence

Each detector record carries the fields {`id`, `manifest_path`, `manifest_yaml`, `policy_id`, `violation_text`}; the embedded `manifest_yaml` decouples downstream stages from the filesystem, so the proposer and verifier operate on the record alone. A representative record and its full schema ship with the released artifact (`data/detections.json`).

The test suite (`make test`) covers detector contracts, proposer guards, verifier gates, scheduler ordering, and patch idempotence, with generated patch-minimality checks gated behind `make artifact-test`. Beyond the deterministic contract tests, `tests/test_property_guards.py` runs property-based checks over hundreds of randomized manifests per run, confirming that the per-policy patchers add the expected hardening (dropping dangerous capabilities, denying privilege escalation, enforcing `RuntimeDefault` seccomp, preferring non-privileged defaults) and remain idempotent without widening the universal verifier gate beyond the invariants of Section 3.

4.2 Dataset and Configuration

Two deliberately vulnerable manifests (`001.yaml`, `002.yaml`) are retained for smoke tests, but all evaluation numbers in this report come from the much larger Grok corpus (5,000 manifests mined from ArtifactHub [26]) and the supported corpus (1,264 manifests curated after policy normalization). `configs/run.yaml` remains the source of truth for proposer mode, retry budgets, and API endpoints. The current opt-in Grok/xAI rerun configuration uses `grok-4.3` against the xAI chat-completions endpoint with temperature 0, seed 1337, a 60 s timeout, and two retries per call; switching out of deterministic rules mode requires the corresponding external credentials. The supplemental

appendix documents the ArtifactHub mining pipeline and corpus hashes, and `ARTIFACTS.md` maps each table, figure, and headline claim to its data source.

The privileged-DaemonSet case study follows the same artifact trail. For a Cilium-style manifest, the proposer preserves required host mounts while changing `privileged` and `allowPrivilegeEscalation` to false, dropping all Linux capabilities by default, and enforcing `RuntimeDefault` seccomp. The example is retained in `docs/privileged_daemonsets.md`; the paper uses it only to illustrate how verifier gates preserve required host integrations while blocking privilege-escalation paths.

Cross-cluster replay used 200 manifests per provider-targeted environment and the same verifier triad plus fixtures. The EKS-targeted slice succeeded on 198/200 manifests with 198/198 accepted patches; the GKE and AKS slices each succeeded on 200/200 with 200/200 accepted patches. We report these as replay outputs (see Table 10); broader multi-provider generalization remains future work. Per-provider `summary.csv` and `results.json` files live under data/cross_cluster/{eks,gke,aks}/, with collection steps in docs/cross_cluster_replay.md.

5 Evaluation

5.1 Research Questions

We organize the evaluation around four research questions. The design makes a specific bet—that verification, not the generator, should be the acceptance boundary—and each question probes one consequence of that bet: whether the loop actually accepts useful fixes (RQ1), whether risk-aware ordering buys anything over the naive queue an operator would otherwise use (RQ2), whether that ordering starves low-risk work (RQ3), and whether the accepted patches are small enough to review and trust (RQ4).

RQ1 (Robustness): does the closed loop accept fixes across corpora and modes? *Rationale:* a remediation loop is only useful if a large fraction of detections survive all four gates, and that fraction should not collapse when the proposer switches from deterministic rules to an LLM backend. *Finding:* the loop delivers 88.52% acceptance on the Grok-5k sweep, 100.00% on the supported 1,264-manifest corpus in rules mode, and 13,338/13,373 (99.74%) accepted on the full 15,718-detection run under deterministic rules plus guardrails (auto-fix rate 0.8486 over detections; median 9 operations), with no `hostPath`-related safety failures remaining.

RQ2 (Scheduling effectiveness): does risk-aware ordering reduce the wait for dangerous items? *Rationale:* operators triage a backlog, so the metric that matters is not throughput—which is bounded by proposer/verifier latency regardless of order—but how long the highest-risk items wait. *Finding:* the risk-aware scheduler ($Rp/\mathbb{E}[t]$ + UCB-style exploration + aging + KEV boost) reduces top-risk P95 wait from 102.3 hours under FIFO to 13.0 hours (7.9×) at comparable throughput.

RQ3 (Fairness): does prioritization starve low-risk work? *Rationale:* any priority queue risks indefinitely deferring low-risk items, so the aging term must be shown to bound starvation rather than merely shift it. *Finding:* aging keeps the median rank of the top-50 high-risk items at 25.5 while still draining lower-risk items; starvation is zero in the bounded parameter sweeps and falls from 93.4% to 19.1% for high-risk items in the longer queue replay.

RQ4 (Patch quality): are the accepted patches small and stable? *Rationale:* a patch that an operator must hand-audit is only reviewable if it is minimal and does not drift under reapplication. *Finding:* generated JSON Patches remain small (median 9 operations on the full 15,718-detection corpus; 5–8 on the smaller curated slices) and idempotent under repeated application.

5.2 Experimental Setup and Results

Unless explicitly noted, results derive from the deterministic `rules` pipeline. Table 4 consolidates acceptance and latency statistics for each corpus. The API-backed Grok mode is likewise benchmarked (4,426 / 5,000 accepted; see data/batch_runs/grok_5k/metrics_grok5k.json) but requires external credentials and funded access, so we treat it as an opt-in configuration rather than the default reproduction path. Consolidated metrics (acceptance + latency) live in data/eval/unified_eval_summary.json. Table 2 states, for each headline rate, the exact unit and corpus it is computed over, so acceptance, auto-fix, and risk figures are not conflated.

Detector wrapper coverage. The detector stage wraps `kube-linter` and policy-engine findings; we do not claim a novel detector. Running `scripts/eval_detector.py` on a synthetic nine-policy hold-out confirms that the wrapper emits the expected finding on each purpose-built fixture. Because each policy has exactly one fixture, this is a regression-coverage check rather than a real-world accuracy estimate. The live replay validates remediation *safety* for the items the detector flags—every accepted patch passed server-side dry-run and live apply checks (data/live_cluster/results_1k.json; summary in data/live_cluster/summary_1k.csv)—but it does not by itself measure detector recall on uncurated manifests.

ArtifactHub structural labels. To avoid scoring against detector-derived labels, we added a separately implemented structural oracle for the 69 checked-in ArtifactHub manifests. It parses YAML fields against configured policy semantics, including absent non-root, read-only-root-filesystem, resource-limit, and capability-drop settings, before scoring detector output. Against these labels, the wrapper records 143 true positives, 19 false positives, and three false negatives: structural-agreement precision 0.883 (95% bootstrap CI 0.843–0.916), structural-agreement recall 0.979 (0.962–1.000), structural-agreement F1 0.929 (0.902–0.948), and structural-agreement weighted F1 0.944 (0.932–0.960). Residual disagreements are concentrated: 16/19 false positives are capability-related policy-boundary cases, two are service-selector cases, one is a Job TTL case, and all three false negatives are service-account references in generated job/cronjob manifests. We therefore treat this as structural-oracle agreement and error-analysis evidence, not a broad detector benchmark; labels are checked in at data/eval/artifacthub_sample_labels_structural.json, with detections, metrics, residual errors, and protocol files mapped in `ARTIFACTS.md`.

Table 2
Metric denominators. Each headline number is computed over a specific unit and corpus; we state these explicitly to avoid ambiguity across acceptance, auto-fix, and risk metrics.

Dataset	Unit	Count	Mode	What the metric measures
Live replay	manifests	1,000	rules + seeded fixtures	server-side dry-run and live apply acceptance
Supported corpus	manifests	1,264	rules	per-manifest acceptance (curated after policy normalization)
Supported risk set	detections	1,278	rules + risk scheduler	policy-weighted risk removal (ΔR)
Full corpus (detections)	detections	15,718	rules + guardrails	total detections found
patched subset	patches	13,373	rules + guardrails	patches attempted
accepted subset	patches	13,338	rules + guardrails	verifier-accepted (99.74% of attempted)
Grok/xAI slice	manifests	1,313	LLM (opt-in)	paired rules-vs-Grok slice acceptance (100.00%)
Grok-5k	manifests	5,000	LLM (opt-in)	LLM patch acceptance (88.52%)

Definitions. "Acceptance" is computed over *attempted* patches; "auto-fix rate" (0.8486) is computed over *all* detections; live-replay "success" means both server-side dry-run and live apply passed for the fixture-seeded manifest; "risk removal" uses policy-weighted detections. The curated corpora are normalized to supported policies, so per-manifest acceptance on those sets is not directly comparable to detection-level rates on the full corpus.

The evaluation campaigns span deterministic and LLM-backed modes. Rules mode repairs 1,264/1,264 supported manifests (100%) with 29 ms median proposer latency and 242 ms median verifier latency (P95 517.8 ms). Historical note: the checked-in extended-5k verifier-record snapshot records 4,677/5,000 accepted patches (93.54%), but because current reruns of the visible verifier differ on the same 5k candidate records, we do not use that count as a current verifier-rerun result. The archived Grok/xAI 5k cache delivers 4,426/5,000 remediations (88.52%) with median JSON Patch ops 9; telemetry records 4.38M input, 0.69M visible completion, and 11.40M total tokens including xAI-reported reasoning tokens, allowing cost to be recomputed from current pricing [10]. That cache is the manuscript source for acceptance and token counts. Later namespaced Grok 4.3 validation artifacts are retained as supplemental evidence rather than replacing the canonical 88.52% result. On the full rules+guardrails run (15,718 detections), the pipeline accepts 13,338/13,373 patched items (99.74%; auto-fix rate 0.8486 over detections) with median patch ops 9. Figure 4 contrasts deterministic throughput with Grok/xAI's API-driven variance.

To ground deployability we instrumented Grok/xAI proposer and verifier traces from a separate archived 200-manifest replay (data/-batch_runs/grok200_latency_summary.csv, data/-batch_runs/verified_grok200_latency_summary.csv). The proposer shows 5.10 s median end-to-end latency (P95 16.9 s), while the verifier adds 89.5 ms median latency (P95 369.3 ms). Failure causes remain dominated by dry-run contract mismatches and legacy StatefulSets; Table 5 summarises the top categories and ties each mitigation to a regression class. These latency medians are not a full-5k latency estimate; full 5k acceptance and token telemetry are reported separately.

The failure taxonomy (Table 5) shows that the largest Grok dry-run failure category (65 cases) comes from the Kubernetes API refusing to return an existing object, often for CRDs needing elevated RBAC; 20 cases come from stale core/v1 resource lookups, and the long tail is dominated by invalid StatefulSet/CronJob specs. These counts shaped the mitigations used uniformly in rules and Grok modes: seed CRD+RBAC fixtures, pre-flight StatefulSets for missing `volumeMounts`, and route immutable-field dry-run errors to human review. Figure 3 compares Kyverno admission-time mutation with post-hoc verified repair, while Figure 5 summarizes the simulated scheduler variants.

Live-cluster replay on a stratified 1,000-manifest subset in the fixture-seeded Kind/Kubernetes 1.34.0 evaluation environment achieves 100.0% server-side dry-run and live apply acceptance (1,000/1,000) with full alignment between the two checks on this subset. The verifier seeds bespoke service accounts and injects benign placeholder images where manifests omit them. Guardrail importance is quantified by a 5k boundary matrix computed from checked-in verifier-record snapshots: weaker policy-only or admission-style boundaries admit 312 rules-mode records and 551 Grok/xAI policy-only records that the corresponding full-verifier snapshots reject (Table 7). The rules-mode matrix inputs are historical verifier records, and current visible-verifier reruns on the same candidate records differ, so the matrix should be read as snapshot-derived guardrail evidence rather than a live implementation rerun.

Risk-aware scheduling reduces queue latency for high-risk items. Using empirical success probability p_i, latency $\mathbb{E}[t_i]$, and risk R_i, the scheduler lowers top-risk P95 wait from 102.3 h (FIFO) to 13.0 h while keeping the median rank of the top 50 items at 25.5. Figure 2 shows the same effect by tier: high-risk work waits less than an hour with risk-aware ordering yet idles for 26–50 h under FIFO. The longer replay preserves the result: high-risk median wait is 14.75 h for risk-aware ordering versus 130.33 h for FIFO, and starvation drops from 93.4% to 19.1%. Risk calibration removes 55,935/56,990 units (98.15%) on the supported set and 227,330/242,300 units (93.82%) on the 5k sweep, with throughput near 4.5–4.9 risk units per expected-time interval (Table 3). Scheduler A/B simulations yield 1,259 assignments per arm and comparable mean risk closed for risk-aware ordering and FIFO (42.97 vs. 43.40); the gain is which items are served first.

The queue replay reports both Gini and starvation because Gini alone is misleading: FIFO's Gini coefficient (0.28) is lower than the risk-aware scheduler's (0.34), yet 93% of high-risk FIFO work waits beyond 24 h versus 19% under risk-aware ordering.

Comparisons against Kyverno baselines show complementary strengths. The live-cluster Kyverno mutate run accepts 364/381 detections (95.54%) once patched manifests pass our verifier, and the mutating webhook exceeds 98% on fixture-compatible overlapping policies while remaining

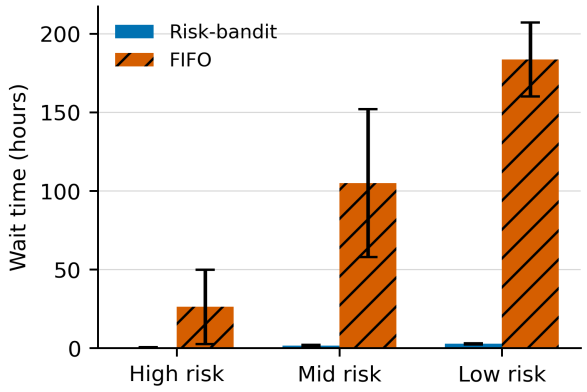


Figure 2. Median and P95 wait time by risk tier for risk-aware and FIFO queue replays.

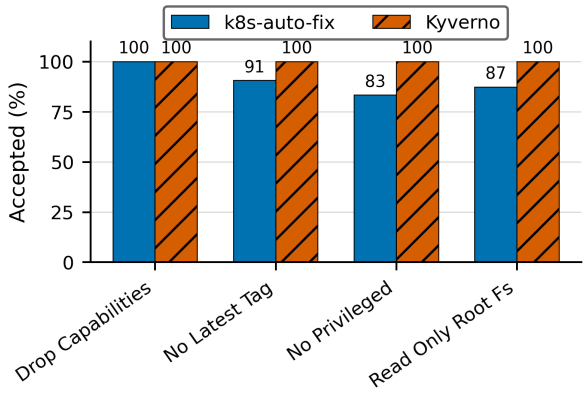


Figure 3. Admission-time Kyverno mutation and post-hoc verified repair on overlapping policies (seed 1337).

lower for policies that require more context. Our deterministic rules+guardrails run reaches 99.74% over attempted patches, and the 19-sample verifier-ablation study reaches 78.9% under full gates while catching four policy escapes. Cross-version simulations retain $> 96\%$ risk reduction.

Reading Tables 3–4. ΔR is risk removed by accepted fixes; residual is risk left; $\Delta R/t$ is risk removed per minute of expected work; P95 is the 95th-percentile time; and auto-fix counts patches accepted automatically without manual edits. The supported row aggregates the curated 1,278-detection rules replay, while "Rules (historical 5k snapshot)" captures the extended 5,000-manifest historical verifier-record snapshot. Risk uses the same scheduler units as Section 5: a privileged pod carries 85 units, missing `runAsNonRoot` carries 70, and a floating `:latest` tag carries 50. Removing 55,935 of 56,990 units therefore retires 98.15% of the supported-corpus blast radius, and the $\Delta R/t$ values (4.49–4.88) feed the risk-aware scheduler baselines.

Detailed per-manifest deltas between rules and Grok/xAI on the 1,313-manifest slice are documented in the project artifact docs/ablation_rules_vs_grok.md. The operator survey instrument is drafted in docs/operator_survey.md; it documents the planned human-in-the-loop rotation listed in Table 10.

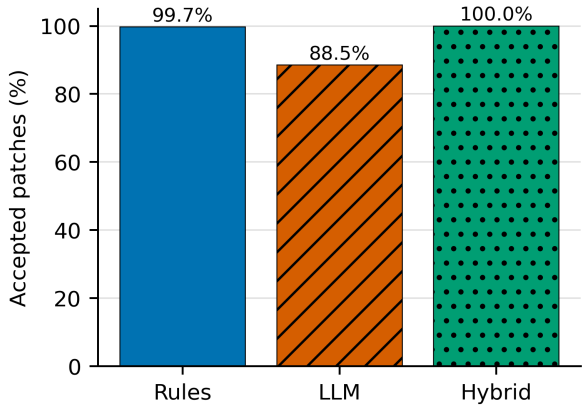


Figure 4. Verifier acceptance across deterministic, LLM-backed, and hybrid proposer modes; the rules-only 5k evidence in this comparison is an archived verifier-record snapshot rather than a current verifier rerun.

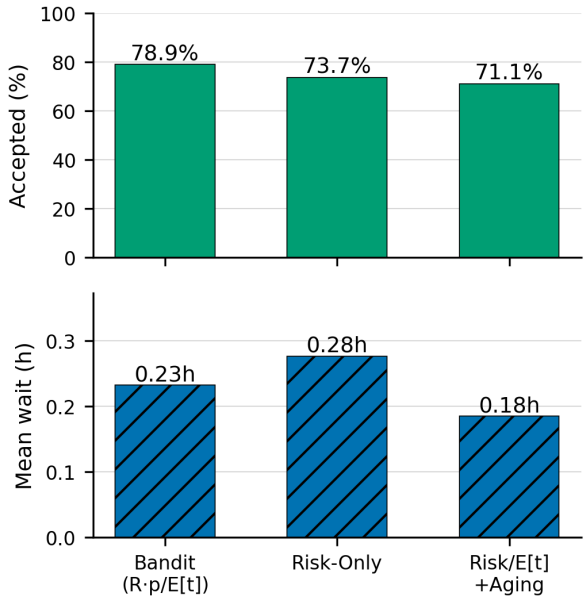


Figure 5. Simulated scheduler acceptance and mean wait over 247 queue assignments.

6 Ablations and Failure Taxonomy

Table 6 shows how fixture seeding collapses the verifier failure categories across the supported corpus.

6.1 Verifier Ablations

Replay of the 830-item queue snapshot (data/metrics_schedule_compare.json) quantifies how each scheduler treats critical detections. All heuristics clear roughly the same workload—$\Delta R/t = 247.2$ risk units per hour—because proposer/verifier throughput dominates. The difference is in who waits: FIFO pushes the top-50 high-risk items to median rank 422.5 (P95 620) and P95 wait 102.3 h, while the risk-only variant (R) and the full risk-aware scheduler (risk, aging, KEV boost, UCB-style exploration) keep the same cohort within median rank 25.5 (P95 48) and cap top-risk P95 wait at 13.0 h. Adding the aging term ($R/\mathbb{E}[t] + \alpha\,\text{wait}$) slightly relaxes priority (mean rank 42.2,

Table 3
Risk calibration summary. $\Delta R/t$ divides risk removed by summed expected-time priors; the 5k row is snapshot-derived rather than a current verifier rerun.

Dataset	Det.	Accepted	ΔR	Residual	ΔR/R	ΔR /t
Supported	1,278	1,259	55,935	1,055	98.15%	4.49
Rules (historical 5k snapshot)	5,000	4,677	227,330	14,970	93.82%	4.88

Table 4
Acceptance and latency summary (seed 1337).

Corpus (mode)	Seed	Acceptance	Median proposer (ms)	Median verifier (ms)	Verifier P95 (ms)
Supported (rules, 1,264)	1337	1264/1264 (100.00%; 95% CI 99.70–100)	29.0	242.0	517.8
Full corpus (rules+guardrails, 15,718 detections)	1337	13338/13373 (99.74%; 95% CI 99.64–99.81; auto-fix 0.8486 over detections)	–	–	–
Manifest slice (Grok/xAI, 1,313)	1337	1313/1313 (100.00%; 95% CI 99.71–100)	–	–	–
Grok-5k (Grok/xAI)	1337	4426/5000 (88.52%; 95% CI 87.61–89.37)	–	–	–

Notes: Manifest counts are in data/eval/table4_counts.csv. Dashes mean row-level latency was not regenerated for that acceptance artifact; the archived Grok 200-manifest telemetry is cited in Section 5. Wilson intervals are inline and tabulated in data/eval/table4_with_ci.csv; multi-seed supported-corpus replays are in data/eval/multi_seed_summary.csv. `scripts/eval_significance.py` rebuilds data/eval/significance_tests.json; Grok server round-trips differ sharply from deterministic verifier latency ($p = 3.2 \times 10^{-47}$).

Table 5
Top 10 API-backed LLM dry-run failure causes by count. Latency medians are reported separately from the 200-manifest trace summaries.

Failure Cause	Count
Batch API lookup returned no current object	65
Core/v1 API lookup returned no current object	20
ReplicationController template validation failed	18
Deployment server-side dry-run rejected the object	16
JSON Patch attempted to remove missing `clusterName`	16
ReplicaSet server-side dry-run rejected the object	14
StatefulSet container volume context was invalid	14
Deployment container spec validation failed	12
CronJob job-template validation failed (Keras workload)	11
CronJob job-template validation failed (RetinaNet workload)	11

Table 6
Verifier failure taxonomy before and after fixture seeding.

Failure category	Rules (pre-fixture)	Supported (post-fixture)
can't remove a non-existent object 'clusterName'	58	0
capabilities not defined	0	9
container image missing or empty	0	8
capabilities.drop missing	0	6
privileged container detected	0	6
capabilities.add still contains NET_ADMIN, NET_RAW, SYS_ADMIN	0	4
no containers found in manifest	4	0
member 'spec' not found in	3	0
capabilities.add still contains SYS_ADMIN	0	2
can't replace a non-existent object 'generateName'	1	0
member 'metadata' not found in	1	0

P95 124) but preserves the low top-risk wait (13.0 h) needed for fairness.

A finer-grained sweep over exploration and aging coefficients (data/metrics_schedule_sweep.json) shows the risk-aware scheduler sustaining high-risk median wait of 17.3 h (P95 32.8 h) even when exploration weight is set to 1.0, while low-risk items absorb most of the slack (median 120.9 h). The condensed simulation in data/operator_ab/summary_simulated.csv supports the same conclusion on a 247-assignment toy queue: the 152 risk-aware assignments close 78.9% of tasks with mean wait 0.23 h and P95 0.91 h, while the companion risk-only and aging-baseline rows report the alternate scheduling modes used for the comparison.

Table 7 quantifies the acceptance boundary on the shared 5k rules and Grok/xAI verified-record snapshots. The matrix holds candidate-patch records fixed and weakens the acceptance predicate: accepting every emitted patch, accepting only records that clear the original scanner/policy assertion, accepting records that clear the policy and recorded API-admission flag, and finally applying the corresponding full-verifier snapshot. Here the API-admission column is the stored `ok_schema` flag, which is produced by `kubectl apply -dry-run=server` when that gate is enabled. The weaker boundaries accept more records, but they also admit records the full-verifier snapshot rejects. On rules-5k, policy-only and admission-style boundaries would accept 312 additional historical snapshot rejects; on Grok-5k, policy-only acceptance would admit 551 dry-run or safety rejects. These snapshot-derived results broaden the earlier 19-sample pilot, while preserving the caveat that absolute counts inherit the verifier configuration and record-generation path that produced each checked-in record and are not presented as current verifier reruns. Figure 4 summarizes acceptance across rules-only, LLM-only, and hybrid modes with the

Table 7
Acceptance-boundary matrix on checked-in 5k verifier-record snapshots. Escapes are records accepted by a weaker boundary but rejected by the corresponding full-verifier snapshot; the API-admission column uses the recorded `ok_schema` server-side dry-run flag. The rules snapshot is historical and not a fresh rerun of the current verifier.

Dataset	Boundary	Accepted	Accept.	Escapes
rules_5k	Ungated proposer output	5000/5000	100.00%	323
rules_5k	Scanner/policy re-check only	4989/5000	99.78%	312
rules_5k	Policy + API-admission gates	4989/5000	99.78%	312
rules_5k	Full verifier boundary	4677/5000	93.54%	0
grok_5k	Ungated proposer output	5000/5000	100.00%	574
grok_5k	Scanner/policy re-check only	4977/5000	99.54%	551
grok_5k	Policy + API-admission gates	4426/5000	88.52%	0
grok_5k	Full verifier boundary	4426/5000	88.52%	0

same archived/snapshot caveat for the rules-only 5k evidence.

Terminology. An *ablation* weakens the recorded acceptance predicate to measure its effect; an *escape* in Table 7 is a record that would be accepted by that weaker predicate but was rejected by the corresponding full-verifier snapshot.

Domain-invariant enforcement vs. generic acceptance. The matrix is not a named-agent comparison and does not assign proxy results to Codex Security, KubeIntellect, or Kyverno. It asks a narrower acceptance-boundary question on existing verified records: what would be admitted if the verifier stopped at weaker recorded evidence? For deterministic rules, the domain policy and recorded API-admission flags both pass on 4,989/5,000 records, but the full-verifier snapshot accepts 4,677. For Grok/xAI patches, policy-only acceptance would admit 4,977/5,000 records, while adding the recorded API-admission flag drops acceptance to the full-verifier result of 4,426/5,000 because malformed or dry-run-rejected patches overlap the remaining safety failures. The earlier 19-sample no-policy pilot still records four concrete capability-hardening escapes (`ids 007, 012, 013, 016`); the 5k matrix is larger snapshot-derived corroboration that weaker recorded boundaries trade higher apparent acceptance for records the corresponding full-verifier snapshots rejected. A like-for-like run against a named agent on identical manifests remains the future-work item in Section 9.

6.2 Metrics and Measurement

We measure effectiveness with auto-fix rate, no-new-violations rate, patch minimality, time-to-patch, risk reduction, throughput, and fairness. Auto-fix rate is accepted patches divided by detected violations; no-new-violations rate is accepted patches with zero new policy, API-admission, or universal-safety violations divided by accepted patches; patch minimality is median JSON Patch operations; and time-to-patch reports P50/P95 from enqueue to acceptance. Risk reduction uses $\Delta R_i = R_i^{\text{pre}} - R_i^{\text{post}}$ and reports both $\sum_i \Delta R_i$ and rate. Throughput is accepted patches per hour. Fairness reports P95 wait by risk tier plus starvation: the fraction of items waiting more than 24 h before scheduling. The supplemental appendix gives a worked risk example.

Full-run summary. Running the full corpus of 15,718 detections in rules+guardrails mode yields 13,338 accepted out of 13,373 patched items (99.74%; auto-fix rate 0.8486 over detections) with a median of 9 JSON Patch operations and 35 safety failures (all non-hostPath edge cases). In deterministic queue replay, risk-aware ordering gives top-risk items P95 wait of 13.0 h at roughly 6.0 patches/hour while FIFO defers the same cohort to 102.3 h (+89.3 h).

Targets (Acceptance Criteria). We require auto-fix rate $\geq 70\%$, no-new-violations rate $\geq 95\%$ under the full verifier, and median JSON Patch operations ≤ 6 on curated rules-mode smoke sweeps (median 5, P95 6). Detector hold-out results are wrapper regression coverage only; ArtifactHub structural labels provide bounded agreement evidence. The full-corpus rules+guardrails replay reaches median patch ops 9 because multi-violation manifests receive several independent hardening edits, and the weaker-boundary rows in Table 7 intentionally violate the no-new-violations target.

7 Related Work

Legend (Table 8). The risk-aware scheduler balances risk, fairness, and expected time using UCB-style exploration; "guardrails" are the safety checks (policy, schema, dry-run) plus simple sanitization before apply; "JSON Patch" is a small, surgical edit to the YAML; and "CRD/fixtures" are supporting Kubernetes objects the API expects (namespaces, service accounts, custom resources) that some admission tools require before they can mutate a file.

Metric caveat. Table 8 aggregates metrics reported by prior work that span admission latency, MTTR, and acceptance rates, so values are not strictly comparable; they provide qualitative context only.

Table 9 grounds those differences with shared per-policy slices, not a single aggregate leaderboard. Tables 8 and 9 therefore use different rosters on purpose: Table 8 compares representative systems by lifecycle role, while Table 9 is limited to tools with reusable policy-level artifacts or scanner interfaces for the same manifest slices. `k8s-auto-fix` lands 33–100% acceptance across targeted high-risk policies, while unsupported `no_host_ports` remains 0%. Some cells have small denominators, and Kyverno/Polaris rows use the larger detection sets their policies can process; we therefore treat the table as role/coverage evidence. Kyverno's mutate CLI reaches 100% on overlapping checks when fixtures are present, but admission can fail before mutation when they are absent. Polaris' CLI is detection-oriented, MAP depends on the Kubernetes CEL/JSONPatch admission-policy surface, and aligned LLMSecConfig prompts accept none of this slice. We do not read these results as "outperforming" Kyverno; our distinction is post-hoc repair of *existing* manifests plus per-patch verifier evidence.

We group prior work into five categories—detection-only tooling, admission-time policy engines, LLM-based configuration repair, large-scale SRE automation, and emerging security agents—and state, for each, the gap that motivates a closed verification loop. Table 8 summarizes the comparison; the discussion below makes the categorization explicit.

1. Detection-only tooling. Static analyzers such as `kube-linter` [5] and dashboards such as Polaris [6] identify misconfigurations and score manifests against best-practice checks; GLITCH shows the broader value of

Table 8
Lifecycle comparison of Kubernetes remediation systems (May 2026 source snapshot).

Capability	k8s-auto-fix (this work)	GenKubeSec [21]	Kyverno [23]	Borg/SRE [25]
Primary Goal	Closed-loop hardening (detect→patch→verify→prioritize)	LLM-based detection/remediation suggestions	Admission-time policy enforcement	Large-scale auto-remediation in production clusters
Fix Mode	JSON Patch (rules + optional LLM)	LLM-generated YAML edits	Policy mutation/generation	Custom controllers and playbooks
Guardrails	Policy re-check + schema + `kubectl apply -dry-run=server` + privileged/secret sanitization + CRD seeding	Manual review; no automated gates	Validation/mutation webhooks; assumes controllers	Health checks, automated rollback, throttling
Risk Prioritization	Risk-aware scheduler ($Rp/\mathbb{E}[t]$ + UCB-style exploration + aging + KEV boost)	Not implemented	FIFO admission queue	Priority queues / toil budgets
Evaluation Corpus	15,718 detections (rules+guardrails: 13,338/13,373 patched = 99.74%; auto-fix 0.8486; median ops 9); 1,000 live-cluster manifests (100.0% server-side dry-run and live apply acceptance, seeded fixtures); 5,000 Grok manifests (88.52%); 1,264 supported manifests (100.00% rules, curated)	≈277k labeled KCFs; precision 0.990, recall 0.999 vs. RB tools; 30-sample expert validation of explanations/remediation (Malul et al.)	Admission-time policies and examples; no public comparable post-hoc repair corpus or acceptance percentage in the cited docs	Millions of production workloads (no public acceptance %)
Telemetry	Policy-level success probabilities, latency histograms, failure taxonomy	Token/cost estimates; no pipeline telemetry	Admission-controller reports and policy outcomes; no comparable proposer/verifier telemetry in the cited docs	MTTR, incident counts, operator feedback
Outstanding Gaps	Infrastructure-dependent rejects, operator study, scheduled guidance refresh in CI	Automated guardrails, risk-aware ordering	LLM-aware patching, risk-aware scheduling	Declarative manifest fixes, static analysis integration

Table 9
Per-policy coverage on shared security-context slices. Denominators differ by tool and policy; the table provides coverage evidence rather than a single aggregate leaderboard.

Policy	k8s-auto-fix	Kyverno	Polaris (CLI)	Polaris (webhook)	MAP	LLMSecConfig
drop_cap_sys_admin	2/3 (66.7%)	n/a	n/a	n/a	1/3 (33.3%)	0/3 (0.0%)
drop_capabilities	1/2 (50.0%)	12/12 (100.0%)	0/12 (0.0%)	0/12 (0.0%)	1/2 (50.0%)	0/4 (0.0%)
env_var_secret	n/a	3/3 (100.0%)	0/3 (0.0%)	0/3 (0.0%)	n/a	n/a
no_host_path	1/1 (100.0%)	n/a	n/a	n/a	0/1 (0.0%)	0/1 (0.0%)
no_host_ports	0/1 (0.0%)	n/a	n/a	n/a	0/1 (0.0%)	0/1 (0.0%)
no_latest_tag	1/2 (50.0%)	25/25 (100.0%)	0/25 (0.0%)	0/25 (0.0%)	1/2 (50.0%)	0/2 (0.0%)
no_privileged	1/3 (33.3%)	5/5 (100.0%)	0/5 (0.0%)	0/5 (0.0%)	0/1 (0.0%)	0/1 (0.0%)
read_only_root_fs	2/3 (66.7%)	107/107 (100.0%)	0/107 (0.0%)	86/107 (80.4%)	1/3 (33.3%)	0/3 (0.0%)
run_as_non_root	2/3 (66.7%)	63/63 (100.0%)	0/63 (0.0%)	37/63 (58.7%)	1/3 (33.3%)	0/3 (0.0%)
set_requests_limits	4/6 (66.7%)	285/285 (100.0%)	0/285 (0.0%)	135/285 (47.4%)	1/3 (33.3%)	0/6 (0.0%)

technology-agnostic IaC security-smell detection [33]. Their output, however, is a finding rather than a validated, ready-to-apply patch. We reuse such detectors as the *Detector* stage and add the missing repair-and-verify stages on top.

2. Admission-time policy engines. Kubernetes admission control runs mutating logic before validating logic [3]; Kyverno [23] and OPA Gatekeeper [4] validate or mutate manifests at admission time, including Kyverno strategic-merge/JSONPatch and mutate-existing rules [24]. They are powerful guardrails, but they act primarily on the create/update path into the cluster and do not, by themselves, repair the large body of *existing*, non-compliant resources or rank outstanding work by risk; mutation also depends on the cluster already carrying the namespaces, service accounts, and CRDs an incoming manifest references. Our verifier reuses these engines for the policy re-check while operating

post-hoc on stored manifests.

3. LLM-based configuration repair. GenKubeSec [21] uses LLM prompts to detect, localize, and explain Kubernetes misconfigurations and to suggest remediations; it reports strong detection precision/recall (we cite the authors' figures and did not reproduce their pipeline) but leaves the application and validation of fixes manual. Minna *et al.* mine Artifact Hub Helm charts, compare Checkov/KICS findings, and evaluate LLM-generated mitigations on the same ecosystem our corpus draws from [19]; Lian *et al.* study LLM-based configuration validation across systems [20]. LLMSecConfig [18] generates LLM repairs guided by scanner findings and policy identifiers, but its acceptance check is scanner-level and does not include a server-side dry-run or the universal safety invariants we enforce. These works make LLMs useful candidates; the security-reasoning limits observed by Ullah *et al.* motivate treating model output as untrusted until external verification succeeds [31].

4. Large-scale SRE automation. Production fleet-management systems such as Borg [25] automate remediation and rollback at scale, but they target running infrastructure and operational toil rather than the static hardening of application manifests before deployment.

5. Emerging security agents. Concurrent with this work, OpenAI describes Codex Security as a research-preview application-security agent [29], while KubeIntellect [30] and KubeLLM [22] propose LLM-orchestrated agents for Kubernetes management and troubleshooting. These agents target general software vulnerabilities or broad cluster operations; neither is centered on enforcing Kubernetes-specific configuration invariants (schema, policy re-check, and server-side dry-run) as the acceptance boundary for each patch. Codex Security is available as a research preview rather than as a public batch-evaluation interface for identical-manifest experiments, so a direct experimental comparison is not yet possible; we position against it qualitatively and leave a head-to-head agent comparison to future work in Section 9.

Positioning. Across these categories, prior tools cover individual stages—detect, block, or suggest—but rarely close the loop. The distinguishing acceptance boundary in `k8s-auto-fix` is that existing manifests, deterministic patches, and LLM-generated patches all pass through the same multi-gate verifier (policy re-check, server-side dry-run/API admission, and universal safety invariants) before accepted work is ordered by auditable policy risk rather than first-in-first-out. This emphasis follows a broader APR lesson: scanner- or test-passing patches can still be overfit or unsafe unless the acceptance boundary checks the domain property the patch was meant to preserve [34]–[37]. All reported results are reproducible from the released data and scripts.

SAST vs. DAST positioning. Many tools only scan manifests offline (SAST). Our verifier additionally exercises a live API server with `kubectl apply -dry-run=server` [9] (DAST), which submits the request to the server without persisting it, so cluster-specific problems—missing CRDs, namespaces, or quotas—are caught before apply. This static-to-API acceptance path is what the live-cluster evaluation in Section 5 stresses.

8 Limitations

The system prioritizes verifier evidence over autonomous deployment, so several constraints remain. **External validity** is bounded by corpora that skew toward Helm-derived workloads; we refresh the ArtifactHub scrape monthly, add partner manifests as available, and will use the drafted operator survey to supplement deterministic replays. **Detector validity** is scoped: the wrapper delegates to `kube-linter` and policy-engine findings, and the separately implemented ArtifactHub structural oracle is a single-annotator, Helm-rendered slice rather than a broad detector benchmark. **Fixture sensitivity** remains because server dry-runs need namespaces, service accounts, CRDs, and related objects that mirror production; the fixture harness now installs common dependencies and records misses for future cluster-specific bundles. **LLM limits** include latency, cost, provider drift, and hallucinated fields; every LLM patch is therefore cached, telemetry-backed, and gated by the same verifier triad and semantic-regression checks as rules-mode patches. **Verification scope** is limited to policy, API-admission, and universal safety checks, not full workload semantic equivalence; snapshot-derived ablations also inherit the verifier version and gate configuration that generated each checked-in record. The scheduler cold-starts policies without prior telemetry from configured defaults (policy risk or 40 risk units, success probability 0.9, expected time 10.0, zero wait, and KEV flags for selected high-risk policies) until policy metrics computed from verifier outcomes replace those defaults. High-risk semantic edits such as non-allowlisted `hostPath` rewrites, dropped capabilities, or read-only-root-filesystem changes with unknown runtime needs should route through human review unless the workload requirement is known. **Experimental validity** is bounded by deterministic scheduler replays, simulated operator A/B results, replayed cross-cloud outputs, and the lack of a public Codex Security batch interface for head-to-head agent comparison.

9 Conclusion and Future Work

The closed-loop verifier records 100.0% server-side dry-run and live apply acceptance on a fixture-seeded live-cluster replay (1,000/1,000 stratified manifests; Tables 9 and 4, data/live_cluster/results_1k.json). Offline, the supported 1,264-manifest slice records 100.00% acceptance in rules mode, the 1,313-manifest Grok/xAI slice records 100.00%, Grok-5k records 88.52%, and rules + guardrails accept 13,338 / 13,373 patched items (99.74%; auto-fix 0.8486 over 15,718 detections) with median patch ops 9 (Table 4, data/metrics_rules_full.json). In deterministic queue replay, the risk-aware scheduler trims top-risk P95 wait from 102.3 h (FIFO) to 13.0 h (data/metrics_schedule_compare.json).

The paper-facing summary tables regenerate from checked-in JSON/CSV artifacts by `make reproducible-report` (this target re-renders from checked-in artifacts and does not re-execute proposers or verifiers); live replay, cross-cluster replay, LLM API reruns, and figures are mapped in `ARTIFACTS.md`. Scheduler comparisons stem from deterministic queue replays rather than live analyst rotations. The gains are anchored in deterministic guardrails, server-side dry-run/API-admission enforcement, and universal safety checks, with

Table 10
Evidence status. We distinguish results that are completed from those that are replay/simulation-based or planned, so that claims are not read as more than they are.

Evidence	Status	Scope
Rules pipeline	Completed	Deterministic corpora (1,264 supported; 15,718-detection full run)
Grok/xAI run	Completed (opt-in)	Credentialed external API; 5,000-manifest sweep
Live replay	Completed	Fixture-seeded single cluster, 1,000 manifests
Cross-cloud replay	Completed (replay outputs)	200 manifests each on EKS/GKE/AKS-labeled targets; provider environment recorded in the artifact, not independently re-verified here
Scheduler comparison	Replay-based	Deterministic queue replays, not a live operator deployment
Operator A/B	Replay/simulated only	247-assignment simulation plus 1,259/arm staging replay; not a live operator study
Human-in-the-loop rotation	Planned	Survey instrument drafted; future work

matching xAI/Grok runs available when credentials and budget are supplied from the published token counts and current pricing [10]. Every accepted patch is surfaced through a GitOps helper that records verifier evidence, captures the JSON Patch diff, and requires human approval before merge [7], [11]. Auto-apply should stay off for image substitutions, required `hostPath` paths, read-only-root-filesystem or dropped-capability changes with unknown runtime needs, and regulated namespaces; those cases route to review.

Future work will fold EPSS and CISA KEV feeds directly into R_i, add batch-aware fairness policies, and replace the simulated scheduler A/B with a live human-in-the-loop rotation. We will also broaden seeded fixtures and Kyverno webhook baselines across new corpora, then run head-to-head agent comparisons: KubeIntellect is the near-term public candidate, while Codex Security comparison awaits a public interface or API suitable for identical-manifest batch evaluation. That experiment would connect Kubernetes remediation to closed-loop repair benchmarks for general software patches [17].

Artifact availability. The complete artifact—source, evaluation scripts, datasets, telemetry, and the `make reproducible-report` entry point—is public at github.com/bmendonca3/k8s-auto-fix; the submitted revision should use the repository commit packaged with the manuscript. All reported metrics were generated with random seed `1337` on Python 3.12.4 (`kubectl` 1.34.1, `kube-linter` 0.7.6, `kind` 0.30.0; Kubernetes 1.34.0). The optional Grok/xAI proposer requires external credentials and is cached so that artifact evaluation does not depend on live API access; checked-in JSON/CSV data and generated figures support the reported tables and figures without rerunning that API path.

References

[1] CIS Kubernetes Benchmarks. Accessed: May 2026. [Online]. Available: https://www.cisecurity.org/benchmark/kubernetes

[2] Kubernetes: Pod Security Standards. Accessed: May 2026. [Online]. Available: https://kubernetes.io/docs/concepts/security/pod-security-standards/

[3] Kubernetes: Admission Control. Accessed: May 2026. [Online]. Available: https://kubernetes.io/docs/reference/access-authn-authz/admission-controllers/

[4] OPA Gatekeeper How-to. Accessed: May 2026. [Online]. Available: https://open-policy-agent.github.io/gatekeeper/website/docs/howto/

[5] *kube-linter* Documentation. Accessed: May 2026. [Online]. Available: https://docs.kubelinter.io/

[6] Fairwinds, "Polaris: Validation of best practices in your Kubernetes clusters," GitHub repository. Accessed: May 2026. [Online]. Available: https://github.com/FairwindsOps/polaris

[7] Kubernetes: Configure a Security Context. Accessed: May 2026. [Online]. Available: https://kubernetes.io/docs/tasks/configure-pod-container/security-context/

[8] RFC 6902: JSON Patch, DOI:10.17487/RFC6902. Accessed: May 2026. [Online]. Available: https://www.rfc-editor.org/info/rfc6902

[9] `kubectl apply` Command Reference (server-side dry-run). Accessed: May 2026. [Online]. Available: https://kubernetes.io/docs/reference/kubectl/generated/kubectl_apply/

[10] xAI, "API Pricing." Accessed: May 2026. [Online]. Available: https://docs.x.ai/developers/pricing

[11] Kubernetes: Seccomp and Kubernetes. Accessed: May 2026. [Online]. Available: https://kubernetes.io/docs/reference/node/seccomp/

[12] NIST National Vulnerability Database (NVD). Accessed: May 2026. [Online]. Available: https://nvd.nist.gov/

[13] CISA Known Exploited Vulnerabilities Catalog. Accessed: May 2026. [Online]. Available: https://www.cisa.gov/known-exploited-vulnerabilities-catalog

[14] FIRST Exploit Prediction Scoring System (EPSS). Accessed: May 2026. [Online]. Available: https://www.first.org/epss/

[15] Trivy: Vulnerability Scanner for Containers and IaC. Accessed: May 2026. [Online]. Available: https://aquasecurity.github.io/trivy/

[16] Grype: A Vulnerability Scanner for Container Images and Filesystems. Accessed: May 2026. [Online]. Available: https://github.com/anchore/grype

[17] SWE-bench Verified (background on closed-loop code repair evaluation). Accessed: May 2026. [Online]. Available: https://openai.com/index/introducing-swe-bench-verified/

[18] Z. Ye, T. H. M. Le, and M. A. Babar, "LLMSecConfig: An LLM-Based Approach for Fixing Software Container Misconfigurations," in *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*, 2025.

[19] F. Minna, F. Massacci, and K. Tuma, "Analyzing and Mitigating (with LLMs) the Security Misconfigurations of Helm Charts from Artifact Hub," *Empirical Software Engineering*, vol. 30, article 132, 2025, doi: 10.1007/s10664-025-10688-0.

[20] X. Lian, Y. Chen, R. Cheng, J. Huang, P. Thakkar, M. Zhang, and T. Xu, "Configuration Validation with Large Language Models," *arXiv preprint arXiv:2310.09690*, 2023.

[21] E. Malul, Y. Meidan, D. Mimran, Y. Elovici, and A. Shabtai, "GenKubeSec: LLM-Based Kubernetes Misconfiguration Detection, Localization, Reasoning, and Remediation," *arXiv preprint arXiv:2405.19954*, 2024.

[22] M. De Jesus, P. Sylvester, W. Clifford, A. Perez, and P. Lama, "LLM-Based Multi-Agent Framework For Troubleshooting Distributed Systems," in *Proc. 2025 IEEE Cloud Summit*, 2025, pp. 110–115, doi: 10.1109/Cloud-Summit64795.2025.00024.

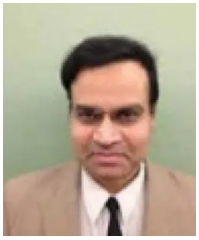
[23] Kyverno Project, "Kyverno Documentation," Accessed: May 2026. [Online]. Available: https://kyverno.io/docs/

[24] Kyverno Project, "Mutate Rules," Accessed: May 2026. [Online]. Available: https://kyverno.io/docs/policy-types/cluster-policy/mutate/

[25] A. Verma *et al.*, "Large-scale Cluster Management at Google with Borg," in *Proc. EuroSys*, 2015; supplemental SRE updates accessed

May 2026. [Online]. Available: https://research.google/pubs/pub43438/
[26] ArtifactHub Documentation. Accessed: May 2026. [Online]. Available: https://artifacthub.io/docs/
[27] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time Analysis of the Multiarmed Bandit Problem," *Machine Learning*, vol. 47, no. 2, pp. 235–256, 2002.
[28] M. Joseph, M. Kearns, J. H. Morgenstern, and A. Roth, "Fairness in Learning: Classic and Contextual Bandits," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2016.
[29] OpenAI, "Codex Security: now in research preview," 2026. [Online]. Available: https://openai.com/index/codex-security-now-in-research-preview/
[30] M. S. Ardebili and A. Bartolini, "KubeIntellect: A Modular LLM-Orchestrated Agent Framework for End-to-End Kubernetes Management," *arXiv preprint arXiv:2509.02449*, 2025.
[31] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini, "LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet?): A Comprehensive Evaluation, Framework, and Benchmarks," *arXiv preprint arXiv:2312.12575*, 2024.
[32] M. S. Islam Shamim, F. A. Bhuiyan, and A. Rahman, "XI Commandments of Kubernetes Security: A Systematization of Knowledge Related to Kubernetes Security Practices," in *Proc. 2020 IEEE Secure Development Conf. (SecDev)*, 2020, pp. 58–64, doi: 10.1109/SecDev45635.2020.00025.
[33] N. Saavedra and J. F. Ferreira, "GLITCH: Automated Polyglot Security Smell Detection in Infrastructure as Code," in *Proc. 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2022, article 47, 12 pages, doi: 10.1145/3551349.3556945.
[34] C. S. Xia, Y. Wei, and L. Zhang, "Automated Program Repair in the Era of Large Pre-trained Language Models," in *Proc. 2023 IEEE/ACM 45th Int. Conf. on Software Engineering (ICSE)*, 2023, pp. 1482–1494, doi: 10.1109/ICSE48619.2023.00129.
[35] J. Petke, M. Martinez, M. Kechagia, A. Aleti, and F. Sarro, "The Patch Overfitting Problem in Automated Program Repair: Practical Magnitude and a Baseline for Realistic Benchmarking," in *Companion Proc. 32nd ACM Int. Conf. on the Foundations of Software Engineering (FSE Companion)*, 2024, pp. 452–456, doi: 10.1145/3663529.3663776.
[36] W. Yang, L. Song, and Y. Xue, "Rust-lancet: Automated Ownership-Rule-Violation Fixing with Behavior Preservation," in *Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE)*, 2024, pp. 1034–1046, doi: 10.1145/3597503.3639103.
[37] I. Bouzenia, P. Devanbu, and M. Pradel, "RepairAgent: An Autonomous, LLM-Based Agent for Program Repair," in *Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE)*, 2025, doi: 10.1109/ICSE55347.2025.00157.
[38] R. Shu, X. Gu, and W. Enck, "A Study of Security Vulnerabilities on Docker Hub," in *Proc. 7th ACM Conf. on Data and Application Security and Privacy (CODASPY)*, 2017, pp. 269–280, doi: 10.1145/3029806.3029832.

Brian Mendonca is an M.S. student at the Georgia Institute of Technology (2024–2026) focusing on secure DevOps, policy-driven remediation, and developer tooling. He received the B.E. in Mechanical Engineering (summa cum laude, GPA 3.99) from Arizona State University in 2021. His prior engineering roles at BAE Systems, Tube Specialties Inc., and BD span aerospace and biomedical quality, Lean/Six Sigma improvement, CAPA, and risk-based quality systems. His research interests include secure configuration management for cloud-native systems, infrastructure-as-code analysis, and data-informed quality engineering.



Vijay K. Madisetti is Professor of Cybersecurity and Privacy at the Georgia Institute of Technology. He earned his Ph.D. in Electrical Engineering and Computer Sciences from the University of California at Berkeley. Professor Madisetti is a Fellow of the IEEE and a recipient of the ASEE Terman Medal. He has authored widely referenced textbooks on cloud computing, data analytics, blockchain, and microservices, and has extensive experience in secure system architectures and privacy-preserving technologies.